

Sustainable Software Engineering

JUnit-to-JMH

What are we going to do today?

Convert JUnit classes into JMH classes

<https://github.com/alniclas/junit-to-jmh>

Useful Links

JU2JMH Paper

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9813593>

JUnit Tests for Performance Testing Paper

[https://dl.acm.org/doi/pdf/10.1145/3377811.3380351?](https://dl.acm.org/doi/pdf/10.1145/3377811.3380351?casa_token=YZtQv6q1UYgAAAAA:BKzp5LBiKLiUpF8q5_6Dq5duoTojHCSii1og4ydSbFJI_gEHZF3c7-0Yh6L6AI0UFvm1O-kE_KkL)

https://dl.acm.org/doi/pdf/10.1145/3377811.3380351?casa_token=YZtQv6q1UYgAAAAA:BKzp5LBiKLiUpF8q5_6Dq5duoTojHCSii1og4ydSbFJI_gEHZF3c7-0Yh6L6AI0UFvm1O-kE_KkL

Why do we use Microbenchmarking?

MICROBENCHMARKING: unit-level benchmarking activity.

Classical performance testing activity involve running **large-scale long-running** tests

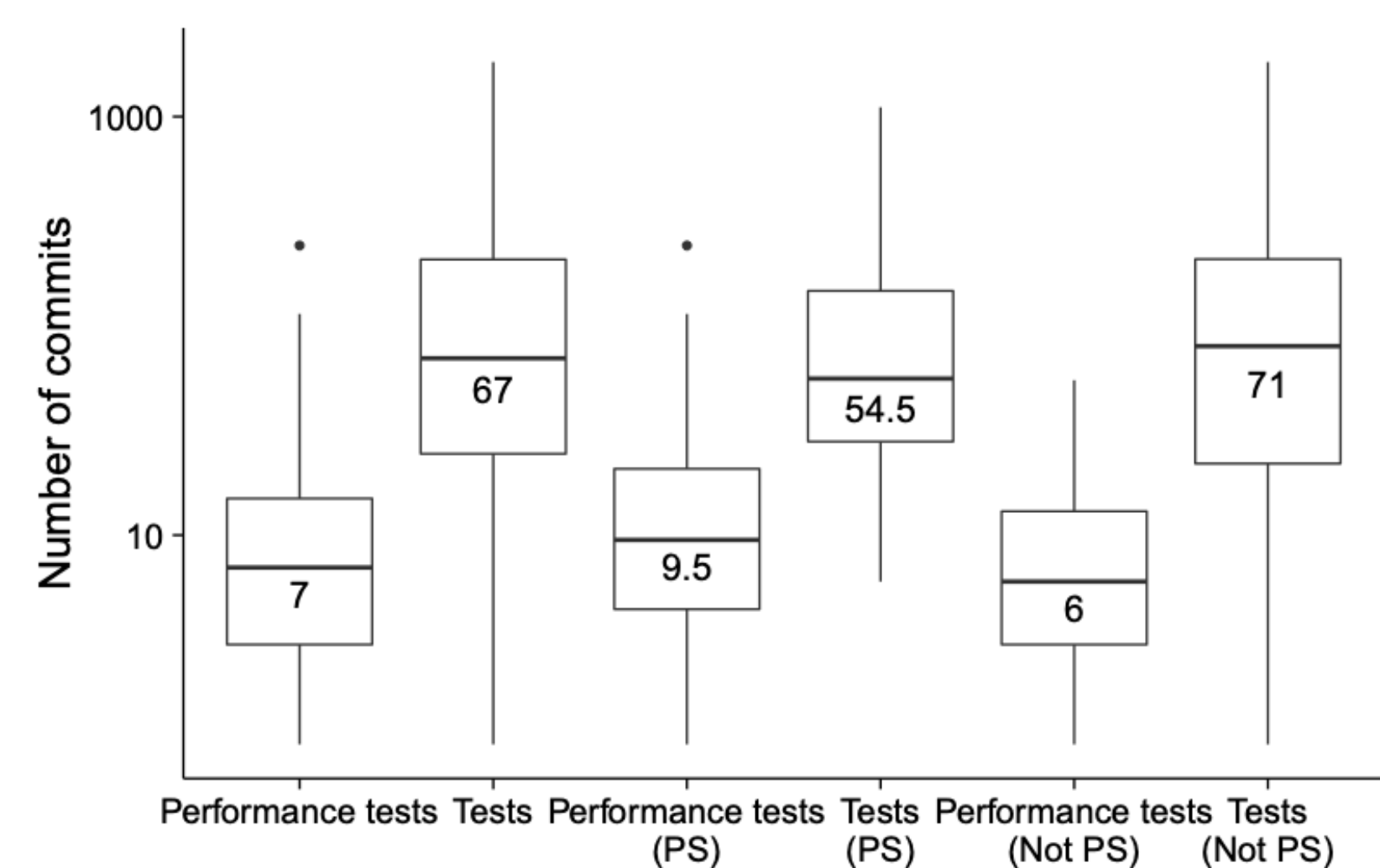
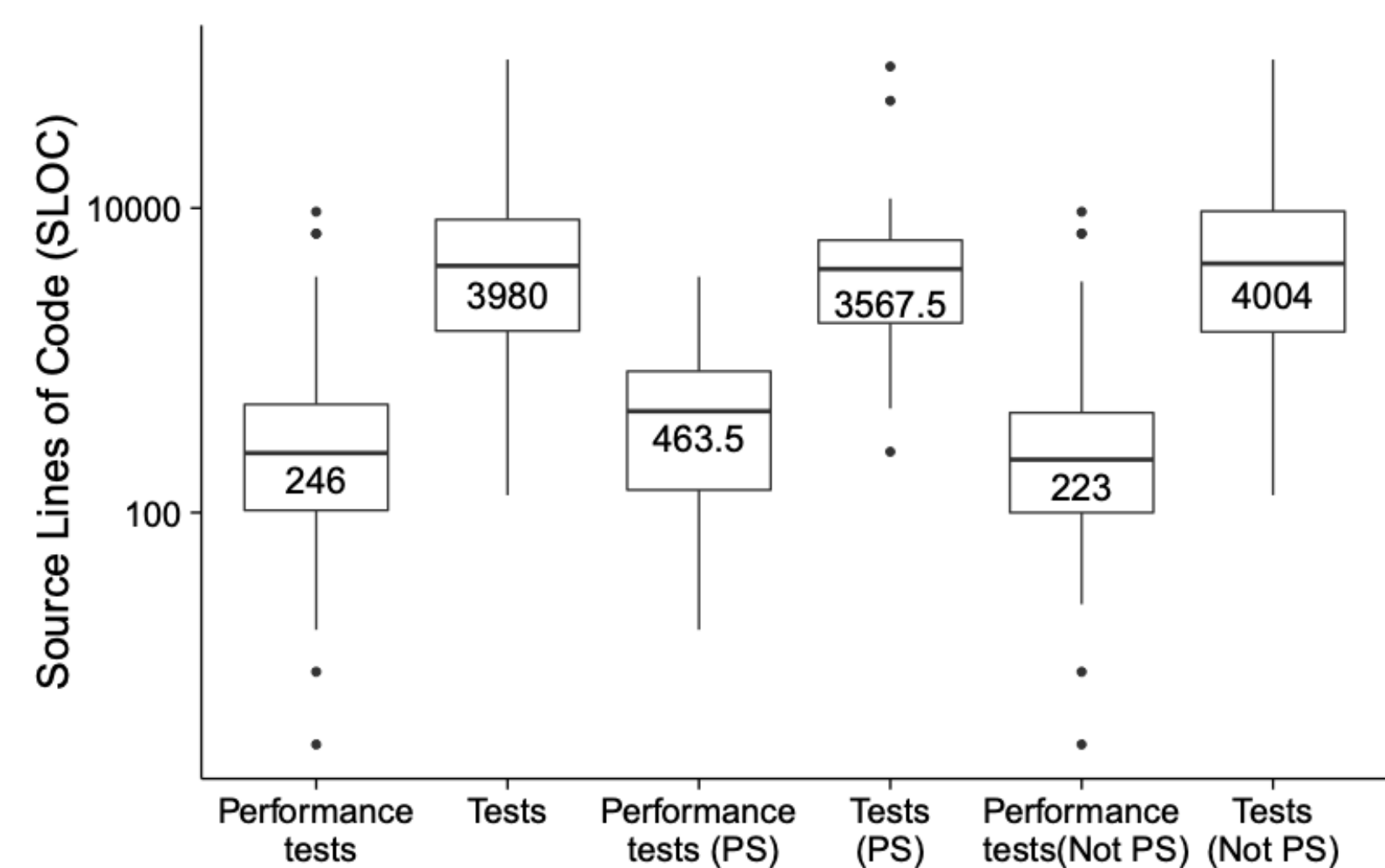
This kind of activities of course are: **time-consuming** and **impossible** to employ in common **DevOps** practices like **CI**

Microbenchmarks represent the solution to these issues, enabling **continuous monitoring** of systems performance

Question: do developers really implement microbenchmarks?

Do Developers Really Implement Microbenchmarks?

Short Answer: Not much as they should.



Question: Why?

Why do Developers Write Few Microbenchmarks?

Microbenchmarks suffer from many design and configuration challenges

- Is easy to stumble in **bad design practices** (e.g., RETU, LOOP, FINAL, INVO, FORK)
- The **configuration** is even more challenging due to JVM optimizations, assuring **quality** is a significant concern
- **Microbenchmarks** can easily become ineffective due to many **environmental** causes

Question: can we use JUnit methods as performance tests?

Can We Use JUnit Methods as Performance Tests?

Short Answer: MAYBE.

From a recent study **it seems** that **most of performance issues** can be **demonstrated** using **readily available tests**

But, only a **small portion** of these tests can be used to find such issues

Still, JMH is **preferable** to directly using JUnit, since it enables **accurate** and **detailed** configurations of microbenchmarks

JU2JMH: a framework to automatically obtain a JMH performance test suite from JUnit tests.

The JU2JMH Approach

```
public class MultiReaderFastListAsReadUntouchableTest {  
  
    @Override  
    @Test  
    public void serialization() {  
        MutableCollection<Integer> collection=this.getCollection();  
        Assert.assertFalse(collection instanceof Serializable);  
    }  
  
}
```

- JU2JMH **identifies** the individual JUnit methods, then **detects** any other **feature** of the tests required
- The JU2JMH **superclass instantiate** and **access** a JUnit class instance, **invokes fixture methods**, **handles expected exceptions**, **executes** the benchmarks
- Copying the tests enables **flexibility**

```
public class MultiReaderFastListAsReadUntouchableTest {  
    @Override  
    @Test  
    public void serialization() {  
        MutableCollection<Integer> collection=this.getCollection();  
        Assert.assertFalse(collection instanceof Serializable);  
    }  
    public static class _Benchmark extends JU2JmhBenchmark {  
        @org.openjdk.jmh.annotation.Benchmark  
        public void serialization() {  
            this.createImplementation();  
            this.runBenchmark(this.implementation::  
                serialization, (this.description("serialization")));  
        }  
        MultiReaderFastListAsReadUntouchableTest implementation;  
        @java.lang.Override  
        public void createImplementation() throws Throwable{  
            this.implementation =  
                new MultiReaderFastListAsReadUntouchableTest();  
        }  
        @java.lang.Override  
        public MultiReaderFastListAsReadUntouchableTest implementation () {  
            return this.implementation;  
            new MultiReaderFastListAsReadUntouchableTest();  
        }  
    }  
}
```

Are JU2JMH Microbenchmarks Effective? And What About LLMs?

- Lack of an **updated** and **exhaustive** study about the **effectiveness** of JUnit methods as microbenchmarks
- Lack of an **exhaustive taxonomy** of **performance ant-patterns** (we're working on it)
- Lack of an **exhaustive** study about the **effectiveness** of **LLMs-generated** JMH microbenchmarks
- And what about a **comparison** between JU2JMH and LLM-generated JMH microbenchmarks